

5.4.6 Implementation with the classical BLAS

The Basic Linear Algebra Subprograms (BLAS) are an interface to commonly used fundamental linear algebra operations. In this section, we illustrate how the unblocked and blocked Cholesky factorization algorithms can be implemented in terms of the BLAS. The explanation draws from the entry we wrote for the BLAS in the Encyclopedia of Parallel Computing [39].

5.4.6.1 What are the BLAS?

The BLAS interface [25] [15] [14] was proposed to support portable high-performance implementation of applications that are matrix and/or vector computation intensive. The idea is that one casts computation in terms of the BLAS interface, leaving the architecture-specific optimization of that interface to an expert.

5.4.6.2 Implementation in Fortran

We start with a simple implementation in Fortran. A simple algorithm that exposes three loops and the corresponding code in Fortran are given by

```
for j := 0, ..., n - 1
  alpha_jj := sqrt(alpha_jj)
  for i := j + 1, ..., n - 1
    alpha_ij := alpha_ij / alpha_jj
  end
  for k := j + 1, ..., n - 1
    for i := k, ..., n - 1
      alpha_ik := alpha_ik - alpha_ij * alpha_kj
    endfor
  endfor
endfor
```

Notice that Fortran starts indexing at one when addressing an array.

Next, exploit the fact that the BLAS interface supports a number of "vector-vector" operations known as the Level-1 BLAS. Of these, we will use

```
dscal( n, alpha, x, incx )
```

which updates the vector x stored in memory starting at address x and increment between entries of $incx$ and of size n with αx where α is stored in $alpha$, and

```
daxpy( n, alpha, x, incx, y, incy )
```

which updates the vector y stored in memory starting at address y and increment between entries of $incy$ and of size n with $\alpha x + y$ where x is stored at address x with increment $incx$ and α is stored in $alpha$. With these, the implementation becomes

```
for j := 0, ..., n - 1
  alpha_jj := sqrt(alpha_jj)
  alpha_{j+1:n-1,j} := alpha_{j+1:n-1,j} / alpha_jj
  for k := j + 1, ..., n - 1
    alpha_{k:n-1,k} := alpha_{k:n-1,k} - alpha_{k,j} * alpha_{k:n-1,j}
  endfor
endfor
```

Here $\alpha_{j+1:n-1,j} = \begin{pmatrix} \alpha_{j+1,j} \\ \vdots \\ \alpha_{n-1,j} \end{pmatrix}$.

The entire update $A_{22} := A_{22} - a_{21}a_{21}^T$ can be cast in terms of a matrix-vector operation (level-2 BLAS call) to

```
dsyr( uplo, n, alpha, x, A, ldA )
```

which updates the matrix A stored in memory starting at address A with leading dimension ldA of size n by n with $\alpha x x^T + A$ where x is stored at address x with increment $incx$ and α is stored in $alpha$. Since both A and $\alpha x x^T + A$ are symmetric, only the triangular part indicated by $uplo$ is updated. This is captured by the below algorithm and implementation.

```
for j := 0, ..., n - 1
  alpha_jj := sqrt(alpha_jj)
  alpha_{j+1:n-1,j} := alpha_{j+1:n-1,j} / alpha_jj
  for k := j + 1, ..., n - 1
    alpha_{k:n-1,k} := alpha_{k:n-1,k} - alpha_{k,j} * alpha_{k:n-1,j}
  endfor
endfor
```

Notice how the code that cast computation in terms of the BLAS uses a higher level of abstraction, through routines that implement the linear algebra operations that are encountered in the algorithms.

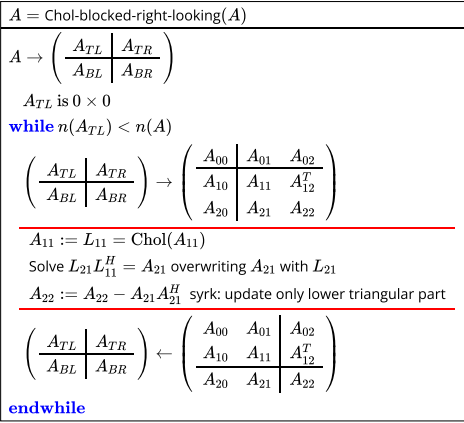


Figure 5.4.6.1. Blocked Cholesky factorization Variant 3 (right-looking) algorithm. The operation "syrk" refers to "symmetric rank-k update", which performs a rank-k update (matrix-matrix multiplication with a small "k" size), updating only the lower triangular part of the matrix in this algorithm.

Finally, a blocked right-looking Cholesky factorization algorithm, which casts most computation in terms of a matrix-matrix multiplication operation referred to as a "symmetric rank-k update" is given in [Figure 5.4.6.1](#). There, we use FLAME notation to present the algorithm. It translates into Fortran code that exploits the BLAS given below.

```

do j=1, nb ,n
  jb = min( nb, n-j )
  Chol( jb, A( j, j ) );

  dtrsm( "Right", "Lower triangular", "Transpose",
         "Unit diag", jb, n-j-jb+1, 1.0d00, A( j,j ), LDA,
         A( j+jb, j ), LDA )

  dsyrk( "Lower triangular", n-j-jb+1, jb, 1.0d00,
         A( j+jb,j ), LDA, 1.0d00, A( j+jb, j+jb ), LDA )
enddo

```

The routines dtrsm and dsyrk are level-3 BLAS routines:

- The call to dtrsm implements $A_{21} := L_{21}$ where $L_{21}L_{11}^T = A_{21}$.
- The call to dsyrk implements $A_{22} := -A_{21}A_{21}^T + A_{22}$, updating only the lower triangular part of the matrix.

The bulk of the computation is now cast in terms of matrix-matrix operations which can achieve high performance.